



Doc 8.1 — `if / else` & Conditional Logic



এই Doc-এ কী শিখব?

JavaScript program অনেক সময় situation অনুযায়ী different কাজ করবে।

যেমন:

- বয়স 18 বা তার বেশি হলে allow করা
- login করা থাকলে dashboard দেখানো
- user admin হলে extra permission দেওয়া
- password ভুল হলে error দেখানো
- cart empty হলে checkout বন্ধ রাখা

এই ধরনের decision নেওয়ার জন্য JavaScript-এ **Conditional Statements** ব্যবহার করা হয়।

এই Doc-এ আমরা শিখব:

1. Conditional Logic কী
 2. Boolean Conditions
 3. `if`
 4. `else`
 5. `else if`
 6. Multiple Conditions
 7. Comparison Operators
 8. Logical Operators
 9. Truthy / Falsy Values
 10. Nested `if`
 11. `else if` vs Nested `if`
 12. Multiple Conditions কীভাবে evaluate হয়
 13. Early Return
 14. Guard Clauses
 15. Real-world Examples
 16. Common Mistakes
 17. Practice Questions
 18. Coding Challenges
 19. Interview Questions
 20. Quick Revision
-

1. Conditional Logic কী?

Conditional logic মানে:

কোনো **condition** সত্য বা মিথ্যা হওয়ার ওপর ভিত্তি করে কোন **code execute** হবে সেটা ঠিক করা।

Simple example:

```
const age = 20;

if (age >= 18) {
  console.log("You can vote");
}
```

এখানে condition:

```
age >= 18
```

age যদি 20 হয়:

```
20 >= 18
↓
true
```

তাই **if** block execute হবে।

Output:

```
You can vote
```

2. Basic Mental Model

if -কে এভাবে ভাবতে পারো:

Condition

↓

true?

/ \

Yes

No

↓

↓

if

skip

আর `if/else` :

Condition

↓

true?

/ \

Yes

No

↓

↓

if

else

3. `if` Statement

Basic syntax:

```
if (condition) {  
    // code  
}
```

Example:

```
const age = 25;  
  
if (age >= 18) {  
    console.log("Adult");  
}
```

Condition true হলে block-এর code execute হবে।

4. `if` Block

Curly braces:

```
{  
    // code  
}
```

এগুলোকে block বলা হয়।

Example:

```
if (true) {  
    console.log("Hello");  
    console.log("Welcome");  
}
```

দুটো statement-ই execute হবে।

5. `if` -এর Condition অবশ্যই Boolean হতে হবে?

JavaScript internally condition-কে Boolean context-এ evaluate করে।

তাই সরাসরি Boolean:

```
if (true) {  
    console.log("Run");  
}
```

এছাড়াও number:

```
if (10) {  
    console.log("Run");  
}
```

এখানে `10` truthy।

String:

```
if ("hello") {  
  console.log("Run");  
}
```

এটাও truthy।

6. Truthy / Falsy Recap

JavaScript-এর কিছু value condition-এ `false` হিসেবে behave করে।

Falsy values:

```
false  
0  
-0  
0n  
""  
null  
undefined  
NaN
```

বাকি সাধারণ values truthy।

Example:

```
if ("hello") {  
  console.log("Runs");  
}
```

কারণ `"hello"` truthy।

7. `if` with Number

```
const score = 80;

if (score >= 40) {
  console.log("Pass");
}
```

Output:

```
Pass
```

আর:

```
const score = 30;

if (score >= 40) {
  console.log("Pass");
}
```

কিছু print হবে না।

কারণ:

```
30 >= 40
↓
false
```

8. `else`

যদি condition false হয় এবং alternative code execute করতে চাই, তখন `else` ব্যবহার করি।

Syntax:

```
if (condition) {
  // true
} else {
  // false
}
```

Example:

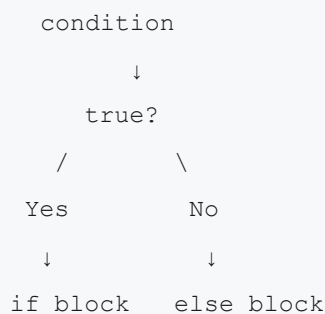
```
const age = 16;

if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

Output:

Minor

9. `if / else` Mental Model



একটা condition-এর জন্য সাধারণত দুইটা possible path।

10. `else if`

যখন multiple conditions check করতে হবে, তখন `else if` ব্যবহার করা হয়।

Example:

```
const score = 75;

if (score >= 80) {
  console.log("A+");
} else if (score >= 70) {
  console.log("A");
} else if (score >= 60) {
  console.log("B");
} else {
  console.log("Fail");
}
```

Output:

A

11. `else if` কীভাবে কাজ করে?

JavaScript conditionগুলো উপর থেকে নিচে check করে।

```
score >= 80 ?
  ↓
  No

score >= 70 ?
  ↓
  Yes
  ↓
Execute A
  ↓
STOP
```

একবার কোনো condition true হলে তার block execute হয় এবং নিচের `else if` আর check হয় না।

12. Order খুব গুরুত্বপূর্ণ

এই code:

```
const score = 85;

if (score >= 40) {
  console.log("Pass");
} else if (score >= 80) {
  console.log("A+");
}
```

Output:

Pass

কেন?

কারণ:

```
85 >= 40
↓
true
```

প্রথম condition-ই true হয়ে গেছে।

তাই `score >= 80` আর check হবে না।

Better:

```
if (score >= 80) {
  console.log("A+");
} else if (score >= 40) {
  console.log("Pass");
}
```

13. Multiple `else if`

তুমি যত প্রয়োজন `else if` ব্যবহার করতে পারো।

```
const temperature = 30;

if (temperature >= 40) {
  console.log("Very Hot");
} else if (temperature >= 30) {
  console.log("Hot");
} else if (temperature >= 20) {
  console.log("Normal");
} else if (temperature >= 10) {
  console.log("Cold");
} else {
  console.log("Very Cold");
}
```

এখানে একটার পর একটা condition check হবে।

14. **else** Optional

else বাধ্যতামূলক না।

এটা valid:

```
if (isLoggedIn) {
  console.log("Dashboard");
}
```

Condition false হলে কিছুই হবে না।

15. **else if** Without **else**

এটাও valid:

```
if (score >= 80) {
  console.log("A+");
} else if (score >= 70) {
  console.log("A");
}
```

কোনোটাই true না হলে কিছু execute হবে না।

16. Comparison Operators with Conditions

Conditional logic-এ comparison operators খুব বেশি ব্যবহার হয়।

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
===	Strict equality
!==	Strict inequality

Example:

```
const age = 20;

if (age >= 18) {
  console.log("Allowed");
}
```

17. Equality Condition

```
const role = "admin";

if (role === "admin") {
  console.log("Admin panel");
}
```

এখানে strict equality ব্যবহার করা হয়েছে।

সাধারণত condition-এ:

```
===

!==
```

ব্যবহার করা ভালো।

18. Multiple Conditions with &&

&& মানে:

সব condition true হতে হবে।

Example:

```
const age = 25;
const hasTicket = true;

if (age >= 18 && hasTicket) {
  console.log("You can enter");
}
```

এখানে:

```
age >= 18 → true
hasTicket → true

true && true
↓
true
```

তাই code execute হবে।

19. Multiple Conditions with ||

|| মানে:

অন্যতঃ একটি condition true হলেই হবে।

Example:

```
const isAdmin = false;
const isModerator = true;

if (isAdmin || isModerator) {
  console.log("Can manage content");
}
```

কারণ:

```
false || true  
↓  
true
```

20. with Conditions

 Boolean value reverse করে।

```
const isLoggedIn = false;  
  
if (!isLoggedIn) {  
  console.log("Please login");  
}
```

এখানে:

```
!false  
↓  
true
```

তাই message দেখাবো।

21. Complex Conditions

Multiple logical operators একসাথে ব্যবহার করা যায়।

```
const age = 25;  
const isLoggedIn = true;  
const isBanned = false;  
  
if (age >= 18 && isLoggedIn && !isBanned) {  
  console.log("Access granted");  
}
```

Condition-এর অর্থ:

18+ হতে হবে

AND

login করা থাকতে হবে

AND

banned হওয়া যাবে না

22. Parentheses ব্যবহার করো

Complex condition readable করার জন্য parentheses ব্যবহার করা ভালো।

```
if ((isAdmin || isModerator) && isLoggedIn) {  
  console.log("Access granted");  
}
```

এখানে:

```
isAdmin OR isModerator  
    ↓  
  result  
    ↓  
AND  
    ↓  
isLoggedIn
```

23. Nested `if`

একটা `if`-এর ভিতরে আরেকটা `if` থাকতে পারে।

```
const isLoggedIn = true;  
const isAdmin = true;  
  
if (isLoggedIn) {  
  if (isAdmin) {  
    console.log("Admin Dashboard");  
  }  
}
```

এখানে আগে login check হচ্ছে।

তারপর admin check হচ্ছে।

24. Nested `if` Real-world Example

```
const isLoggedIn = true;
const hasPermission = true;

if (isLoggedIn) {
  if (hasPermission) {
    console.log("Access granted");
  }
}
```

Mental model:

```
Logged in?
  ↓
Yes
  ↓
Permission আছে?
  ↓
Yes
  ↓
Access granted
```

25. Nested `if` -এর সমস্যা

অনেক বেশি nested condition code-কে difficult করে তোলে।

Example:

```
if (user) {  
  if (user.isLoggedIn) {  
    if (user.isAdmin) {  
      if (user.hasPermission) {  
        console.log("Allowed");  
      }  
    }  
  }  
}
```

এটা deeply nested হয়ে যাচ্ছে।

এ ধরনের code-এ **Guard Clauses / Early Return** ব্যবহার করলে readability অনেক ভালো হয়।

26. **else if** vs Nested **if**

দুটো এক জিনিস না।

else if

```
if (condition1) {  
  // ...  
} else if (condition2) {  
  // ...  
}
```

এখানে **condition1** true হলে **condition2** check হবে না।

Nested **if**

```
if (condition1) {  
  if (condition2) {  
    // ...  
  }  
}
```

এখানে **condition2** কেবল **condition1** true হলে check হবে।

27. Example — `else if`

```
const age = 25;

if (age < 13) {
  console.log("Child");
} else if (age < 18) {
  console.log("Teenager");
} else {
  console.log("Adult");
}
```

এখানে তিনটা mutually exclusive category।

28. Example — Nested `if`

```
const isLoggedIn = true;
const isAdmin = true;

if (isLoggedIn) {
  if (isAdmin) {
    console.log("Admin");
  }
}
```

এখানে second condition প্রথম condition-এর ওপর dependent।

Rule of thumb

Alternative possibilities হলে:

```
if → else if → else
```

Dependent conditions হলে:

```
if
  └─ if
```

29. Conditions Sequentially Evaluate হয়

এই code:

```
const x = 10;

if (x > 100) {
  console.log("A");
} else if (x > 5) {
  console.log("B");
} else {
  console.log("C");
}
```

Evaluation:

```
10 > 100
↓
false

10 > 5
↓
true

B
```

Output:

```
B
```

30. First Matching Condition Wins

এই concept মনে রাখবে:

if → else if → else chain-এ প্রথম true condition-এর block execute হয়।

Example:

```
const age = 25;

if (age >= 18) {
  console.log("Adult");
} else if (age >= 21) {
  console.log("21+");
}
```

Output:

```
Adult
```

যদিও `25 >= 21` true, কিন্তু প্রথম condition already true।

31. Truthy Value দিয়ে `if`

```
const username = "Shariar";

if (username) {
  console.log("Username exists");
}
```

এখানে string non-empty হওয়ায় truthy।

32. Empty String

```
const username = "";

if (username) {
  console.log("Username exists");
} else {
  console.log("Username missing");
}
```

Output:

Username missing

কারণ:

"" → falsy

33. Number Condition

```
const count = 5;

if (count) {
  console.log("There are items");
}
```

5 truthy।

কিন্তু:

```
const count = 0;

if (count) {
  console.log("There are items");
}
```

এখানে execute হবে না।

কারণ:

0 → falsy

Important

যদি 0 valid value হয়, তাহলে blindly truthiness check করা ঠিক নাও হতে পারে।

Instead:

```
if (count > 0) {
  console.log("There are items");
}
```

34. `null` / `undefined` Check

API data-এর ক্ষেত্রে:

```
const user = null;

if (user) {
  console.log(user.name);
} else {
  console.log("No user");
}
```

এখানে `user` falsy।

35. Explicit Boolean Check

যদি variable already Boolean হয়:

```
const isLoggedIn = true;

if (isLoggedIn) {
  console.log("Logged in");
}
```

এটা clean।

অপ্রয়োজনীয়:

```
if (isLoggedIn === true) {
  console.log("Logged in");
}
```

দুটোই কাজ করে, কিন্তু প্রথমটা সাধারণত cleaner।

36. Boolean Condition with `===`

যদি কোনো value সত্যিই নির্দিষ্ট Boolean হতে হবে:

```
if (value === true) {  
  // ...  
}
```

এটা useful হতে পারে যখন `value` arbitrary type-এর হতে পারে।

37. Early Return

Function-এর ভিতরে unnecessary nesting কমানোর জন্য early return খুব useful।

ধরো:

```
function accessDashboard(user) {  
  if (user) {  
    if (user.isLoggedIn) {  
      console.log("Dashboard");  
    }  
  }  
}
```

এটা simplify করা যায়:

```
function accessDashboard(user) {  
  if (!user) {  
    return;  
  }  
  
  if (!user.isLoggedIn) {  
    return;  
  }  
  
  console.log("Dashboard");  
}
```

এখানে invalid cases আগে return করছে।

38. Guard Clause

Guard clause মানে:

Function-এর শুরুতেই invalid/unwanted condition check করে বের হয়ে যাওয়া।

Example:

```
function processOrder(order) {  
  if (!order) {  
    return;  
  }  
  
  if (order.items.length === 0) {  
    return;  
  }  
  
  console.log("Processing order...");  
}
```

এতে main logic নিচে clean থাকে।

39. Guard Clause with Error

```
function withdraw(balance, amount) {  
  if (amount <= 0) {  
    return "Invalid amount";  
  }  
  
  if (amount > balance) {  
    return "Insufficient balance";  
  }  
  
  return "Withdrawal successful";  
}
```

এটা nested `if`-এর চেয়ে readable।

40. Real-world Example — Login

```
const email = "user@example.com";
const password = "123456";

if (!email) {
  console.log("Email is required");
} else if (!password) {
  console.log("Password is required");
} else {
  console.log("Login request can be sent");
}
```

Flow:

```
Email আছে?
  ↓
No → Error

Yes
  ↓
Password আছে?
  ↓
No → Error

Yes
  ↓
Send request
```

41. Real-world Example — Role Based Access

```
const role = "admin";

if (role === "admin") {
  console.log("Full access");
} else if (role === "editor") {
  console.log("Edit access");
} else if (role === "viewer") {
  console.log("Read-only access");
} else {
  console.log("No access");
}
```

42. Real-world Example — Age Validation

```
const age = 17;

if (age < 0) {
  console.log("Invalid age");
} else if (age < 18) {
  console.log("Minor");
} else {
  console.log("Adult");
}
```

থেয়াল করো validation condition আগে এসেছে।

43. Real-world Example — Shopping Cart

```
const cartItems = 3;

if (cartItems === 0) {
  console.log("Your cart is empty");
} else {
  console.log("Proceed to checkout");
}
```

এখানে `=== 0` ব্যবহার করা হচ্ছে কারণ `0`-কে explicitly check করতে চাই।

44. Real-world Example — Stock

```
const stock = 5;

if (stock <= 0) {
  console.log("Out of stock");
} else {
  console.log("Product available");
}
```

45. Real-world Example — Score Grading

```
function getGrade(score) {  
  if (score >= 80) {  
    return "A+";  
  } else if (score >= 70) {  
    return "A";  
  } else if (score >= 60) {  
    return "B";  
  } else if (score >= 50) {  
    return "C";  
  } else if (score >= 40) {  
    return "D";  
  } else {  
    return "F";  
  }  
}
```

এখানে `return` হওয়ার কারণে matching condition-এর পর function শেষ হয়ে যায়।

46. `if` + Function Return

```
function isAdult(age) {  
  if (age >= 18) {  
    return true;  
  }  
  
  return false;  
}
```

এটাকে আরও simple করা যায়:

```
function isAdult(age) {  
  return age >= 18;  
}
```

কারণ:

```
age >= 18
```

নিজেই Boolean result দেয়।

47. **if** Condition-এ Assignment Mistake

একটি common mistake:

```
if (isLoggedIn = true) {  
    // ...  
}
```

এখানে comparison না করে assignment করা হচ্ছে।

Better:

```
if (isLoggedIn === true) {  
    // ...  
}
```

অথবা Boolean হলে:

```
if (isLoggedIn) {  
    // ...  
}
```

48. Another Common Mistake — **=** vs **===**

```
const role = "admin";  
  
if (role = "user") {  
    console.log("User");  
}
```

এটা assignment।

Condition-এর মধ্যে সাধারণত comparison করতে:

```
if (role === "user") {  
  console.log("User");  
}
```

49. Common Mistake — Too Many Nested Conditions

✗ Difficult:

```
if (user) {  
  if (user.isLoggedIn) {  
    if (user.isActive) {  
      if (user.hasPermission) {  
        console.log("Allowed");  
      }  
    }  
  }  
}
```

Better:

```
if (!user) {  
  return;  
}  
  
if (!user.isLoggedIn) {  
  return;  
}  
  
if (!user.isActive) {  
  return;  
}  
  
if (!user.hasPermission) {  
  return;  
}  
  
console.log("Allowed");
```

50. Common Mistake — Wrong Condition Order

✗:

```
if (score >= 40) {  
  console.log("Pass");  
} else if (score >= 80) {  
  console.log("A+");  
}
```

Better:

```
if (score >= 80) {  
  console.log("A+");  
} else if (score >= 40) {  
  console.log("Pass");  
}
```

Rule

More specific / higher threshold conditions সাধারণত আগে রাখো।

51. Common Mistake — Checking length Incorrectly

ধরো:

```
const items = [];
```

এটা:

```
if (items) {  
  console.log("Has items");  
}
```

unexpected হতে পারে, কারণ empty array truthy।

Correct:

```
if (items.length > 0) {  
  console.log("Has items");  
}
```

একইভাবে empty object-ও truthy:

```
if ({} ) {  
  console.log("Runs");  
}
```

52. Common Mistake — 0 Is Falsy

```
const quantity = 0;  
  
if (quantity) {  
  console.log("Quantity exists");  
}
```

কিছু print হবে না।

যদি উদ্দেশ্য হয় quantity zero কিনা check করা:

```
if (quantity > 0) {  
  console.log("Has quantity");  
}
```

53. if vs Ternary

Simple value selection-এর জন্য পরে আমরা ternary ব্যবহার করতে পারব:

```
const message = age >= 18 ? "Adult" : "Minor";
```

কিন্তু complex statements বা multiple actions হলে if/else বেশি readable।

এই Doc-এ মূল focus:

Decision-making logic

Ternary বিস্তারিত থাকবে **Doc 8.2**-তে।

54. `if` vs `switch`

যদি complex ranges/conditions থাকে:

```
if (score >= 80) {  
  // ...  
} else if (score >= 70) {  
  // ...  
}
```

এটা `if/else` -এর জন্ম natural।

আর fixed exact values-এর অনেক case থাকলে `switch` useful হতে পারে।

`switch` বিস্তারিত থাকবে **Doc 8.2**-তে।

55. React Conditional Rendering

React-এ `if` খুব common।

```
function Dashboard({ isLoggedIn }) {  
  if (!isLoggedIn) {  
    return <p>Please login</p>;  
  }  
  
  return <h1>Welcome to Dashboard</h1>;  
}
```

এখানে:

```
isLoggedIn?
```

↓

No → Login message

Yes → Dashboard

56. React Role-based Rendering

```
function AdminPanel({ role }) {  
  if (role !== "admin") {  
    return <p>Access denied</p>;  
  }  
  
  return <div>Admin Panel</div>;  
}
```

এখানে guard clause ব্যবহার হয়েছে।

এটা React code-এ খুব clean pattern।

57. API Data Validation

ধরো:

```
function processUser(user) {  
  if (!user) {  
    return "User not found";  
  }  
  
  if (!user.name) {  
    return "User name missing";  
  }  
  
  return `Hello ${user.name}`;  
}
```

এখানে input validation + guard clause একসাথে ব্যবহার হয়েছে।

58. Multiple Conditions — Practical Example

```
function canCheckout(user, cart) {  
  if (!user) {  
    return false;  
  }  
  
  if (!user.isLoggedIn) {  
    return false;  
  }  
  
  if (cart.length === 0) {  
    return false;  
  }  
  
  return true;  
}
```

এখানে একাধিক condition sequentially validate হচ্ছে।

59. **if** Statement-এর General Pattern

```
if (condition) {  
  // condition true হলে  
} else if (anotherCondition) {  
  // প্রথমটা false, এটা true হলে  
} else {  
  // সব condition false হলে  
}
```

Mental model:

Condition 1

↓

true? — Yes → Block 1

|

No

↓

Condition 2

↓

true? — Yes → Block 2

|

No

↓

else

60. Output Practice

Question 1

```
const age = 20;

if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

Output कौ?

Question 2

```
const score = 85;

if (score >= 90) {
  console.log("A+");
} else if (score >= 80) {
  console.log("A");
} else {
  console.log("Fail");
}
```

Question 3

```
const value = 0;

if (value) {
  console.log("A");
} else {
  console.log("B");
}
```

Question 4

```
const items = [];

if (items) {
  console.log("A");
} else {
  console.log("B");
}
```

Question 5

```
const age = 25;

if (age >= 18) {
  console.log("Adult");
} else if (age >= 21) {
  console.log("21+");
}
```

কোন output হবে এবং কেন?

Question 6

```
const isLoggedIn = false;

if (!isLoggedIn) {
  console.log("Please login");
}
```

61. Coding Challenge 1 — Age Checker

একটি function লিখো:

```
checkAge (age)
```

Rules:

```
age < 0      → "Invalid age"
age < 13     → "Child"
age < 18     → "Teenager"
age >= 18    → "Adult"
```

Example:

```
checkAge (10);
```

Expected:

```
Child
```

62. Coding Challenge 2 — Grade Calculator

Function:

```
getGrade(score)
```

Rules:

```
80-100 → A+  
70-79  → A  
60-69  → B  
50-59  → C  
40-49  → D  
0-39   → F
```

Invalid score হলে:

```
"Invalid score"
```

63. Coding Challenge 3 — Login Validator

Function:

```
validateLogin(email, password)
```

Rules:

```
email missing
  → "Email required"

password missing
  → "Password required"

both available
  → "Login successful"
```

64. Coding Challenge 4 — Access Control

Function:

```
checkAccess(user)
```

Rules:

```
user নেই
  → "No user"

not logged in
  → "Please login"

banned
  → "Account banned"

admin
  → "Full access"

otherwise
  → "Limited access"
```

এখানে **guard clause** ব্যবহার করার চেষ্টা করো।

65. Interview Questions

Q1. `if` statement কী?

Condition true হলে নির্দিষ্ট code block execute করার জন্য ব্যবহৃত conditional statement।

Q2. `else if` কী?

একাধিক alternative condition check করার জন্য ব্যবহৃত হয়।

Q3. `else` কখন execute হয়?

উপরের সব `if` এবং `else if` condition false হলে।

Q4. `else if` chain-এ একাধিক condition true হলে কী হয়?

প্রথম true condition-এর block execute হয়।

Q5. `if` -এ কি শুধু Boolean value দেওয়া যায়?

না। JavaScript value-কে Boolean context-এ evaluate করে।

Q6. Empty array কি truthy?

হ্যাঁ।

```
Boolean([]);
```

ফল:

```
true
```

Q7. Empty object কি truthy?

হ্যাঁ।

```
Boolean({});
```

ফল:

```
true
```

Q8. `0` কি **truthy**?

না।

```
0 → falsy
```

Q9. Nested `if` এবং `else if`-এর পার্থক্য কী?

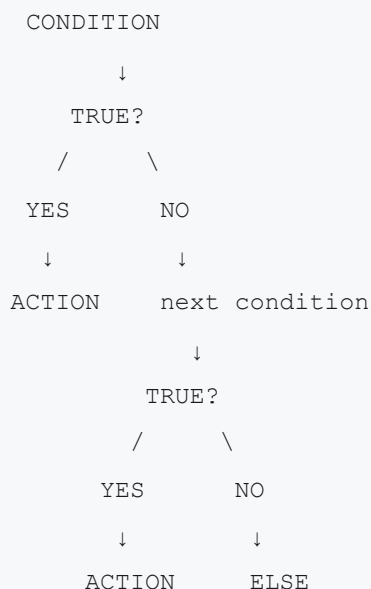
`else if` alternative conditions-এর জন্য, আর nested `if` dependent conditions-এর জন্য বেশি natural।

Q10. Guard clause কী?

Function-এর শুরুতে invalid/unwanted condition check করে early return করার pattern।

66. 🧠 Final Mental Model

Conditional logic পুরোটা এভাবে মনে রাখো:



আর function-এর ক্ষেত্রে:

```
Invalid input?
  ↓
  return

Not authorized?
  ↓
  return

Everything valid
  ↓
Main logic
```

এটাই **Guard Clause mindset**!

67. Golden Rules

1. `if` → condition true হলে code চালায়।
2. `else` → সব previous condition false হলে চলে।
3. `else if` → multiple alternative conditions check করে।
4. `if → else if → else` chain-এ প্রথম true condition win করে।
5. Condition-এর order খুব important।
6. Boolean variables হলে সাধারণত সরাসরি `if (isLoggedIn)` লিখতে পারো।
7. `0`, `""`, `null`, `undefined`, `NaN` falsy।

8. `[]` এবং `{}` truthy।
9. Complex nested condition কমাতে guard clause ব্যবহার করো।
10. Dependent condition → nested `if` হতে পারে।
11. Alternative condition → `else if` বেশি natural।
12. `0` valid value হলে শুধু truthiness দিয়ে check করো না।
13. Array আছে কিনা এবং array-তে item আছে কিনা—দুটো আলাদা:

```
if (items)
```

বনাম

```
if (items.length > 0)
```

14. React-এ early return খুব useful conditional rendering pattern।
15. Complex business logic-এ readability-কে priority দাও।

Quick Revision Table

Concept	Example	Purpose
<code>if</code>	<code>if (age >= 18)</code>	One condition
<code>else</code>	<code>else {}</code>	Alternative
<code>else if</code>	<code>else if (score >= 80)</code>	Multiple alternatives
Nested <code>if</code>	<code>if (user) { if (...) }</code>	Dependent conditions
<code>&&</code>	<code>a && b</code>	Both required
<code> </code>	<code>a b</code>	At least one
<code>!</code>	<code>!isLoggedIn</code>	Reverse condition
Early return	<code>if (!user) return</code>	Exit early
Guard clause	<code>if (invalid) return</code>	Reduce nesting
Truthy check	<code>if (value)</code>	Check existence/presence
Explicit comparison	<code>if (count > 0)</code>	Precise condition

Mastery Checklist

- ☐ Conditional logic বুঝি

- ☐ `if` লিখতে পারি
- ☐ `else` ব্যবহার করতে পারি
- ☐ `else if` chain লিখতে পারি
- ☐ Condition order বুঝি
- ☐ Comparison operators ব্যবহার করতে পারি
- ☐ `&&` condition লিখতে পারি
- ☐ `||` condition লিখতে পারি
- ☐ `!` condition লিখতে পারি
- ☐ Truthy/falsy বুঝি
- ☐ Empty array/object truthy জানি
- ☐ Nested `if` বুঝি
- ☐ Nested `if` কখন avoid করতে হবে জানি
- ☐ Early return বুঝি
- ☐ Guard clause লিখতে পারি
- ☐ React-এ conditional rendering করতে পারি
- ☐ API validation condition লিখতে পারি
- ☐ Role-based access condition লিখতে পারি
- ☐ Login/form validation করতে পারি



Next Doc



Doc 8.2 — `switch`, Ternary & Conditional Expressions

এখানে আমরা শিখব:

```
switch
  ↓
case
  ↓
break
  ↓
default
  ↓
fall-through
  ↓
switch vs if/else
  ↓
ternary ? :
  ↓
nested ternary
  ↓
ternary in React JSX
  ↓
best practices
```

এটা শেষ করলে JavaScript-এ **conditional logic** লেখার **different styles** পরিষ্কার হয়ে যাবে।